

Coding & Solution

Date	28 June 2026
Team ID	*****
Project Name	Smart Agricultural Production Optimization Engine (OptiCrop)
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of the implemented OptiCrop (Smart Agricultural Production Optimization Engine) codebase and overall solution. The submission demonstrates working functionality, clean code practices, and alignment with the defined functional requirements.

Instructions:

- Ensure your code is well-structured, commented, and follows best practices for your chosen technology stack.
- The solution should address the core problem statement and implement the key functional requirements.
- Include all source files, configuration files, and setup instructions needed to run the application.
- Attach or link to your code repository (e.g., GitHub) in the table below.

Solution Summary

Field	Details
Repository Link / URL	[https://github.com/bhavanidunna/OptiCrop]
Programming Language(s)	Python
Framework(s) Used	Flask, Scikit-learn, Pandas, NumPy, Matplotlib, Seaborn, SciPy
Key Features Implemented	• User registration and login (including Google/Facebook sign-in options) • Soil and environmental parameter input form (Nitrogen, Phosphorous, Potassium, temperature, humidity, pH, rainfall) • ML-based crop recommendation engine (Decision Tree, Random Forest, KNN, Gradient Boosting) • Crop suitability and environmental assessment for a user-selected crop • Data visualization of NPK levels, rainfall, and temperature trends • Research/analytics dashboard for aggregated crop-environment patterns
Pending / Incomplete Features	• Downloadable PDF report of crop recommendations • Cloud/hosting deployment of the final application • Refactoring of duplicate input-validation logic across forms • Expanded docstrings and module-level documentation
Setup / Run Instructions	• Clone the repository: <code>git clone [repository-url]</code> • Create and activate a Python virtual environment: <code>python -m venv venv && source venv/bin/activate</code> • Install dependencies: <code>pip install -r requirements.txt</code> • Place the trained model file(s) in the model/ directory • Run the Flask application: <code>python app.py</code>

Field	Details
	• Open the application in a browser at http://localhost:5000

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments:

The crop recommendation engine was trained and evaluated using Decision Tree, Random Forest, KNN, and Gradient Boosting classifiers, with the best-performing model selected for deployment via the Flask interface. Core registration, login, and recommendation flows are functional end-to-end; reporting/export and cloud deployment remain in progress.